

bun_nltk: broad NLTK API coverage in TypeScript with Rust/WASM acceleration and local multi-platform builds

A language-resource toolkit artifact with 241/241
importable names, 432 tests, 91.5 KiB WASM, and three
native targets

Touhidul Alam Seyam

*Department of Computer Science and Engineering, BGC Trust University
Bangladesh, Chattogram, Bangladesh, . ORCID 0009-0007-7512-1893.

Corresponding author(s). E-mail(s): touhidulalam@bgctub.ac.bd;
touhidulalam@bgctub.ac.bd;

Abstract

The Natural Language Toolkit (NLTK) remains a standard teaching and research library, but it is tied to Python. That excludes npm packages, browser code, edge workers, and offline JavaScript clients that cannot run a Python process.

Existing JavaScript libraries such as natural, winkNLP, and compromise implement parts of NLTK. We built **bun_nltk** v0.16.0 to cover more of its API, with selected hot paths in Rust. The package contains native libraries for macOS arm64, Linux x64, and Windows x64, plus a 93,712-byte WebAssembly module. All **241/241 module names** in the NLTK API index import across 46 families. Of these entries, 210 are functional implementations. The remaining 31 comprise 26 shims and five package barrels. The repository records this inventory in an index-derived checklist and a **COVERED** map in `scripts/parity-checklist.py`; 432 tests across 66 files check selected behavior.

On `bench/datasets/gate_synthetic.txt`, using medians from 2 to 4 rounds, Bun with the native library is **12** to **20**× faster than Python on four measured primitives and **34** to **253**× faster on the measured parser and classifier tasks. A separate 15-round comparison with the TypeScript fallbacks finds median speedups of **1.80**× for logistic text training, **1.95**× for SVM training, **1.89**× for HMM Viterbi decoding, **2.54**× for Euclidean K-means, and **2.18**× for WordNet loading. Deterministic 95% percentile-bootstrap intervals remain above **1**× for

all five operations. The interval for a hot WordNet batch crosses $1\times$, so that result is inconclusive. WASM overhead varies by task: about 5.6% for n-gram counting in the dashboard and about 65% for Punkt in the cross-runtime study. Native calls require Bun and a matching prebuilt library. Other hosts use the explicit WASM wrapper. Chromium and Firefox pass the browser harness; Safari, mobile browsers, and edge providers remain untested.

Keywords: Natural Language Toolkit, WebAssembly, Bun, TypeScript, Rust, NLP toolkit, Language Resources, API coverage, Edge computing

1 Introduction

NLTK [1, 2] is a standard reference for natural language processing. Its API spans tokenizers, stemmers, taggers, parsers, language models, WordNet, evaluation metrics, logic, corpora, and other utilities listed at nltk.org/api/nltk.html. Reproducing that range is difficult. NLTK also requires Python, which rules out direct use in npm packages, browsers, Cloudflare Workers and Vercel Edge, Electron and Tauri applications, and offline JavaScript clients. Researchers must either rewrite a baseline in JavaScript or place Python behind a service.

The main JavaScript NLP packages, `natural` [3], `winkNLP` [4], `compromise` [5], and `nlp.js` [6], cover less than 20% of the indexed NLTK modules. They also differ from NLTK on details such as tokenization and stemming. None claims full parity. The API is large, with 241 public modules in 46 families, and it mixes pure algorithms with trained models, GUIs, network clients, corpus downloaders, and external theorem provers.

We chose to port the NLTK interface rather than propose another model. `bun_nltk` v0.16.0 is Apache-2.0 software available at https://github.com/Seyamalam/bun_nltk and on npm as `bun_nltk`. Four properties define the release:

- Import coverage for 241 of 241 names across 46 families in the NLTK API index snapshot, with 210 functional entries and 31 import-only entries. The latter comprise 26 genuine shims and five package barrels.
- 432 tests across 66 files, plus a separate 37-group differential fidelity gate. Selected checks compare Bun output with Python NLTK 3.10.3; shim tests assert the package’s documented error contract rather than equivalence to Python failures.
- On the recorded benchmarks, representative NLP tasks run 12 to $20\times$ faster than Python, while measured parsers and classifiers reach $253\times$ (see §5).

- The npm tarball contains three native targets and a 93,712-byte WASM module. Bun loads the matching native library, while hosts without `dlopen` use an explicit WASM API. Neither path starts Python.

We evaluate the package as language-resource software rather than as a new model. The measurements therefore cover API inventory, behavior, runtime, memory, package size, and reproducibility.

Contributions.

We scraped and enumerated 241 module headings from the NLTK API index and gated import coverage in a local release command. We separated functional implementations from shims, built selected Rust kernels for native and WASM targets, and measured speed, throughput, memory, package size, and cross-platform artifacts. The evaluation reports known output differences instead of treating import coverage as behavioral equivalence.

2 Background and literature review

NLTK and the Python NLP ecosystem.

NLTK [1, 2] defined the teaching API and still anchors most NLP curricula; we use its indexed module headings [7] as the oracle for import coverage. Since 2002 the Python ecosystem has grown around it: spaCy [8] delivers industrial-strength pipelines in Cython for production NER, tagging and parsing; Stanza [9] provides a multilingual neural pipeline covering 70+ languages on Universal Dependencies; and Hugging Face Transformers [10] unifies several hundred pretrained Transformer architectures under one API with a shared model hub. These systems push accuracy ([8–10]) but are not NLTK-compatible by design; they expose different module structures, data schemas and model-loading conventions, remain Python-bound, and cannot be `imported` as drop-in replacements for `nltk.*`.

JavaScript NLP: natural, winkNLP, compromise and nlp.js.

On npm, `natural` [3] covers tokenizers, Porter and Snowball stemmers, Bayes and logistic classifiers, TF-IDF, phonetics, and WordNet. `winkNLP` [4] emphasizes fast tokenization, sentence boundaries, and NER, with reported throughput above 650k tokens/s on an M1. `compromise` [5] uses a lightweight POS and payload tagger with a roughly 14k-word conjugated lexicon. `nlp.js` [6] supplies multilingual NLU/NLG, entity management, and chatbot connectors for more than 40 languages. None claims NLTK parity. For example, `natural` ports Porter and Bayes but omits Punkt [1], CCG, Glue semantics, tableau provers, and BLEU/GLEU/chrF evaluation [7]. `winkNLP` and `compromise` omit chart parsers, WordNet, and translation metrics. `bun_nltk` instead ports the interface, not a model.

Runtimes: Bun, Node, and WebAssembly.

Bun [11] combines a JavaScript runtime, package manager, test runner, and bundler on JavaScriptCore. Its `bun:ffi` module loads native libraries with `dlopen`. WebAssembly [12] provides portable bytecode and a virtual ISA; Haas et al. [13] measured near-native web performance, and the W3C standardized Core Specification v2.0 [12]. Rust [14] compiles the same kernels to native `cdylibs` and a `wasm32-unknown-unknown` module. `cargo-zigbuild` cross-links the Linux and Windows artifacts locally. Earlier NLP/WASM work usually accelerates an isolated kernel such as a tokenizer. This package also includes classifiers, sequence decoding, clustering, and a packed WordNet loader. Cloudflare Workers and Vercel Edge [15] forbid native code, so they use the WASM API rather than Bun’s native bindings.

Language resource evaluation.

Language-resource software calls for evidence about coverage, reproducibility, measured behavior, and known gaps. Sections 3 and 5 report those properties separately rather than reducing them to one parity claim.

3 Methodology

Maintaining import coverage for all 241 indexed names takes a closed loop (Fig. 1).

1. Scrape and enumerate.

A script fetches <https://www.nltk.org/api/nltk.html>, parses its module headings, and emits 241 `nltk.*` names. We froze the denominator from the 2026-08-24 scrape for this release. Future releases must record a new scrape date and denominator because the live documentation can change. This oracle measures indexed names, not complete API or behavioral equivalence.

2. Checklist and COVERED map.

`scripts/parity-checklist.py` diffs the oracle against its `COVERED` inventory and writes `docs/PARITY_CHECKLIST.md` (46 families, 241 entries):

Listing 1 `scripts/parity-checklist.py` inventory excerpt.

```
COVERED: dict[str, bool] = {
    "tokenize": True,
    "stem.porter": True,
    "app": True, # import-only package barrel
    # ... 241 entries
}
```

3. Classify: real port versus shim (Fig. 2).

The decision is shown in `parity_decision.pdf`. A GUI or display-only entry becomes a shim. The same applies when an operation requires a network service or external program that the package does not supply. Portable algorithms use TypeScript,

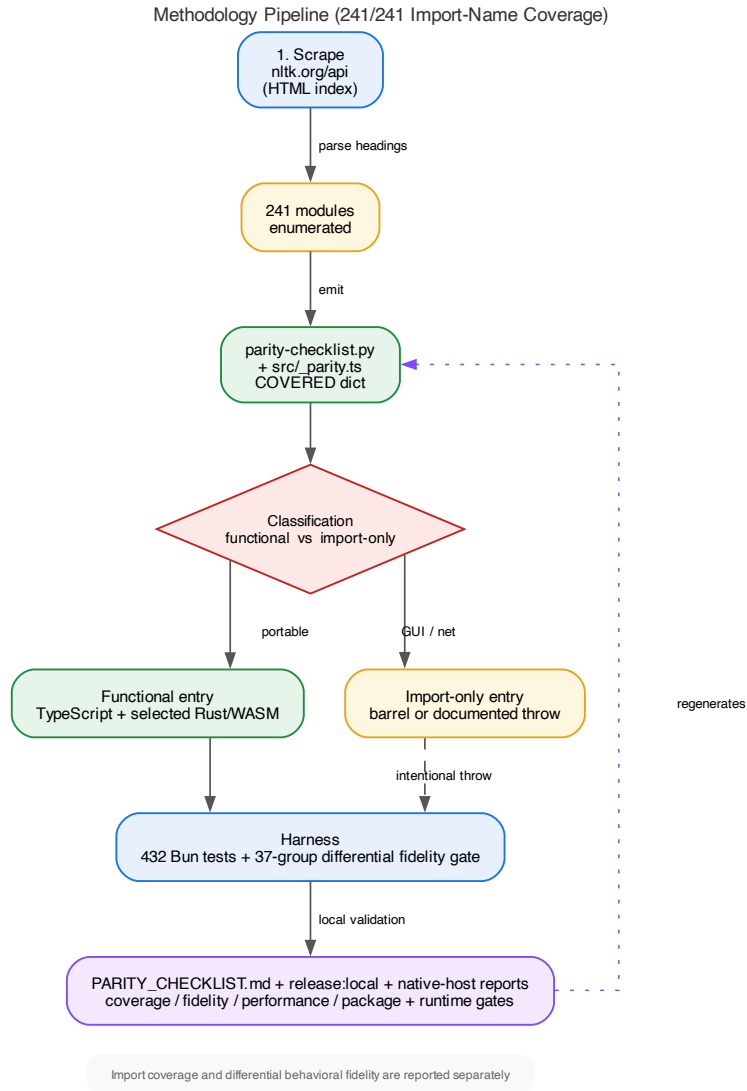


Fig. 1 Methodology pipeline: scrape the NLTK API index, enumerate 241 module headings, emit a checklist and **COVERED** map, classify each entry as functional or import-only, compare selected outputs against Python, then run the local coverage, correctness, performance, size, and package gates.

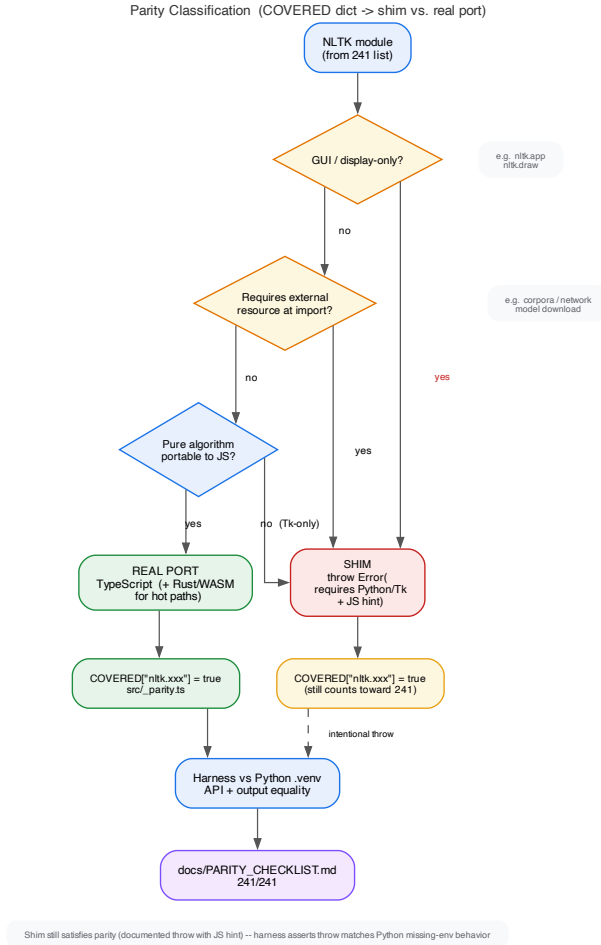


Fig. 2 Coverage classification: the inventory distinguishes functional entries from import-only shims. GUI or display-only modules and entries requiring a network or external program become shims with a documented error. Portable algorithms use TypeScript, with Rust/WASM on selected hot paths. Both categories count toward the 241-name import inventory; only functional entries count as implementations.

with Rust for the measured hot paths (token and n-gram counting, PMI collocations, Porter). The inventory contains 210 functional entries and 31 import-only entries. Five import-only entries are package barrels, leaving 26 genuine shim modules. Shims export named symbols and throw `Error("requires ...; use JS alternative ...")` when unsupported operations are called. Tests assert this specified contract.

They do not claim that Python throws the same exception. Fully shimmed groups include `app`, `draw`, and `twitter`; `chat` mixes re-exports with throwing operations, and `inference.prover9/mace` requires external binaries.

4. Harness: Bun test versus Python .venv.

`bench/` runs selected tasks under Bun and CPython 3.12 with NLTK 3.10.3 on the same data (`bench/datasets/gate_synthetic.txt`). Checks cover API existence and sampled outputs, with stated tolerances for floating-point scores and perplexity. Shim tests check the documented `bun_nltk` error contract. The suite contains 432 tests across 66 files. A separate differential gate runs 37 behavioral groups against live Python NLTK outputs or versioned NLTK-derived fixtures and writes `artifacts/fidelity-report.json`. This gate is separate from the import checklist, but the project author still maintains it. Benchmark rounds report median latency.

5. Local release gate.

`bun run release:local` performs type checking, linting, 34 Rust tests, 432 TypeScript tests, the 37-group differential fidelity gate, native performance thresholds, binary and package-size limits, tarball-content checks, and a packaged macOS runtime smoke. It builds macOS arm64 directly and cross-builds Linux x64 and Windows x64 with Zig, then builds WASM. Companion wrappers run from a fresh clone on x64 Linux and Windows, compare native Rust with the TypeScript fallback on the same host, install the packed tarball, and write versioned JSON reports. The generated checklist proves name coverage against the frozen documentation index; it does not prove complete functional equivalence. None of these procedures requires GitHub Actions.

4 Architecture

Figure 3 traces the package from TypeScript and Rust sources through native and WASM builds to the supported runtimes.

Source: TypeScript plus Rust.

`src/` holds the TypeScript implementation (ESM, strict mode), while `scripts/parity-checklist.py` holds the module-name inventory. `rust/src/` contains the FFI layer and native kernels. Rust handles ASCII token and n-gram work, PMI windows, Porter, Punkt, full text vectorization plus logistic/SVM optimization, HMM Viterbi decoding, Euclidean K-means, and packed WordNet loading and traversal. The text-training call accepts raw documents and returns one binary model buffer, avoiding a token-by-token FFI boundary. WordNet keeps small JSON requests for compatibility but returns tagged binary buffers rather than JSON objects. Other functional entries use TypeScript, and each migrated operation retains a TypeScript fallback for comparison.

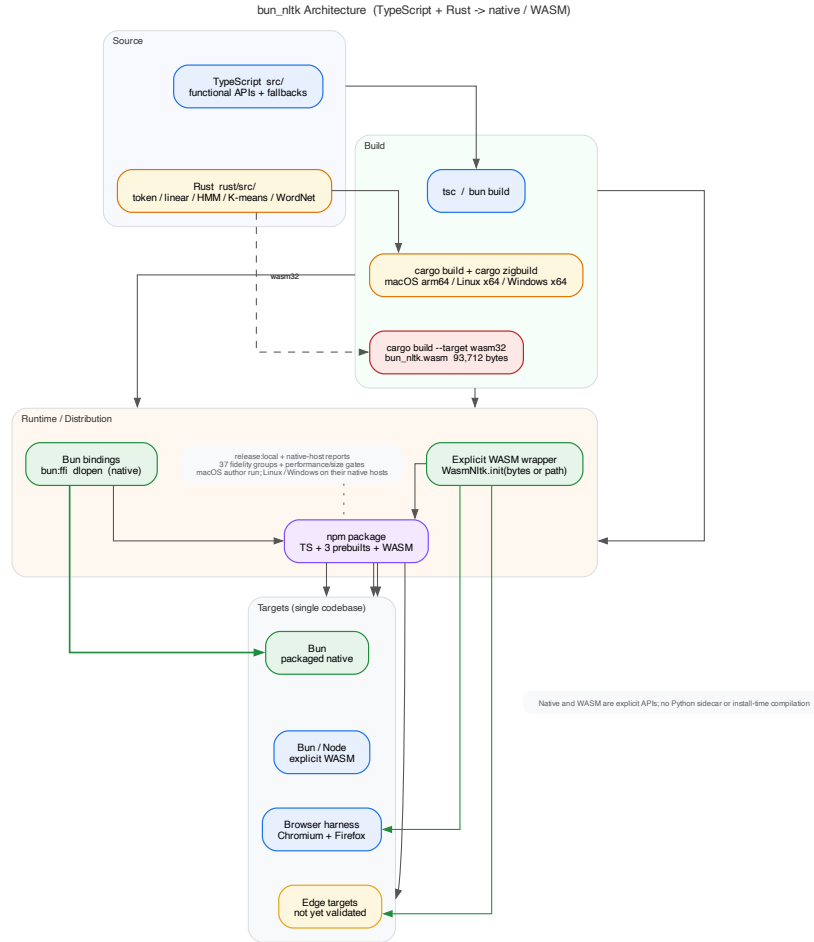


Fig. 3 Architecture: TypeScript APIs call coarse Rust operations for token work, text-model training, HMM decoding, K-means, and WordNet. The local build produces macOS arm64, Linux x64, and Windows x64 libraries plus a 93,712-byte WASM module. Bun loads a compatible native library with `dlopen`; `WasmNltk.init` explicitly instantiates WASM from bytes or a path.

Build: one local command, four binaries.

`bun run release:local` drives the release build. `cargo build` emits the macOS arm64 dylib and WASM; `cargo zigbuild` emits a Linux x64 shared object with a glibc 2.17 baseline and a Windows x64 DLL. Their sizes are 590,080, 728,152, 1,536,512, and 93,712 bytes respectively. The package includes all four. Format checks verify Mach-O arm64, ELF x64, PE x64, and WASM signatures before packing.

Runtime selection.

Listing 2 Explicit native and WASM entry points (simplified).

```
import { countTokensAscii, WasmNltk } from "bun_nltk";

const nativeCount = countTokensAscii(text); // Bun dlopen
const wasm = await WasmNltk.init({ wasmBytes });
const portableCount = wasm.countTokensAscii(text);
wasm.dispose();
```

The root API imports Bun’s native bindings, so native calls require Bun and a matching library. `WasmNltk` is a separate explicit API and accepts caller-supplied bytes for non-filesystem hosts. For the dashboard’s n-gram task, the native median is 0.0741 s and the WASM median is 0.0798 s, so WASM is about 5.6% slower. That result does not generalize to every task: in the M5 Pro study, Punkt takes 1.2 ms native and 2.0 ms in WASM.

Distribution: npm.

`package.json` ships `index.ts`, `src/`, `corpora/`, selected compact models, three native binaries, and WASM. It excludes Rust sources, build scripts, and the 30 MB full WordNet pack. The validated tarball contains 213 files, is 1.72 MB compressed and 4.53 MB unpacked, and has no install-time lifecycle scripts. The local smoke test installs that tarball in a clean project and executes the packaged macOS binary. The release commit also contains the three prebuilts and WASM so a Linux or Windows tester can clone the repository and run one wrapper without a compiler toolchain. Each wrapper packs and installs the tarball on the native operating system before running same-machine native-versus-TypeScript comparisons. Cross-built or emulated runs are not used for cross-platform performance claims.

5 Evaluation

Setup.

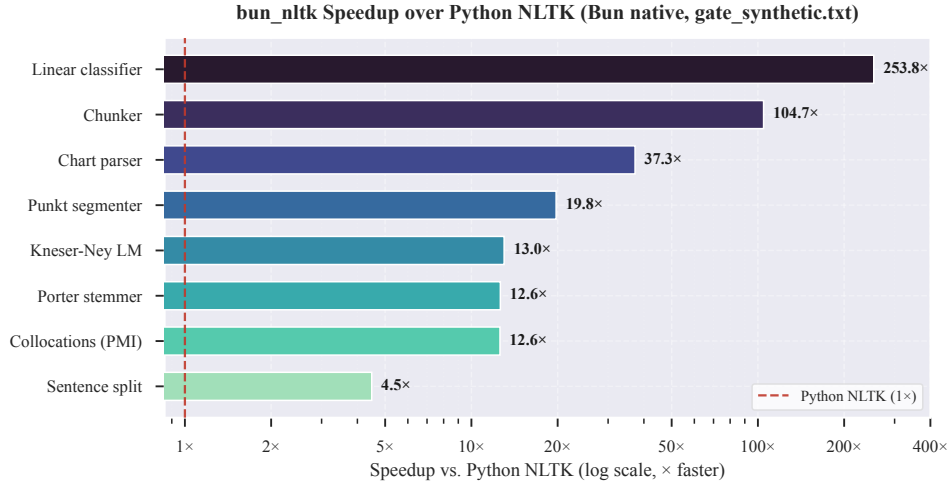
Table 2 uses a recorded single-core Linux x64 runner with CPython 3.12, NLTK 3.10.3, and Bun with the native library. The seeded `bench/datasets/gate_synthetic.txt` file is 7,941,847 bytes, contains 1,148,706 tokens and a 156,675-item vocabulary, and includes about 2.92% non-ASCII text from URLs, handles, and emoji. We ran 2 to 4 rounds per task and report medians without dispersion estimates or confidence intervals. Absolute medians are committed in `artifacts/bench-dashboard.json`; for example, collocations take 1.2999 s in Python and 0.1033 s natively, while Porter takes 3.3024 s and 0.2617 s. Tables 3 and 4 are separate experiments on a MacBook Pro M5 Pro arm64 with NLTK 3.10.3, two warmups, and five measured rounds. Table 1 uses that same Mac for 15-round native-versus-TypeScript comparisons. Its 95% intervals use 10,000 deterministic percentile-bootstrap resamples of the median speedup ratio. We do not pool results across these environments.

Native migration benchmark.

Table 1 measures the operations moved after the original dashboard. Text-classifier timing includes tokenization, vocabulary construction, sparse feature construction, optimization, and holdout evaluation. HMM and K-means compare identical models and outputs. WordNet loading compares the packed Rust backend with parsing and indexing the same payload in TypeScript. The 400-query WordNet batch has a $1.11\times$ median, but its 95% interval is $[0.98, 1.38]$, which crosses $1\times$. We therefore treat hot-batch latency as inconclusive rather than faster.

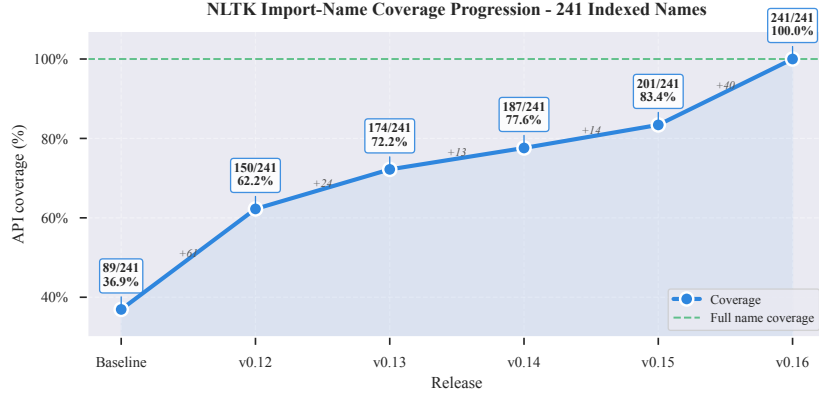
Table 1 Native versus TypeScript fallback, 15 rounds per implementation on an M5 Pro (macOS arm64). Speedup above $1\times$ favors native. Brackets give deterministic 95% percentile-bootstrap intervals from 10,000 resamples. Raw samples are in `paper/bench/native_migration_results.json`.

Operation	Median	95% interval	Check
Logistic text training	$1.80\times$	$[1.75, 1.92]$	Same accuracy
Linear SVM text training	$1.95\times$	$[1.88, 2.12]$	Same accuracy
HMM Viterbi decoding	$1.89\times$	$[1.80, 1.97]$	Same state paths
Euclidean K-means	$2.54\times$	$[2.46, 2.91]$	Centroids within 10^{-9}
Packed WordNet load	$2.18\times$	$[2.07, 2.29]$	Same query checksum
Packed WordNet batch lookup	$1.11\times$	$[0.98, 1.38]$	Interval crosses $1\times$



Dataset: `bench/datasets/gate_synthetic.txt` • median of 2–4 rounds • log-scale x; Python NLTK = $1\times$ baseline

Fig. 4 Speedup over Python NLTK (Bun native, log scale, $\times$ faster; $1\times$ is Python). Representative tasks on the same dataset: collocations/PMI $12.58\times$, Porter $12.61\times$, Punkt $19.75\times$, chunk $104.66\times$, WordNet $101.25\times$, chart parser $37.26\times$, feature chart $147.08\times$, linear classifier $253.75\times$. Data from `artifacts/bench-dashboard.json` (generated 2026-08-21).



Documentation-index names, not behavioral equivalence; snapshot: 2026-08-24; v0.16 imports 241/241 names across 46 families

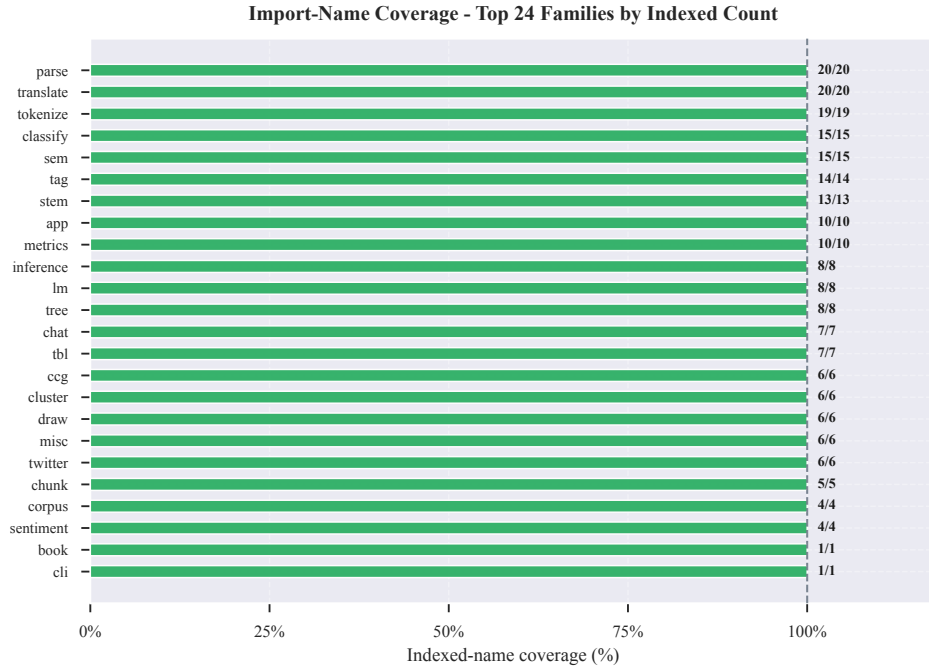
Fig. 5 Import-coverage progression toward the frozen 241-name documentation index. The latest artifact imports 241/241 names across 46 families; this figure does not measure behavioral equivalence.

What we measured.

Table 2 lists every benchmark key in `bench-dashboard.json` as speedup versus Python, median. Figure 4 shows the representative subset on a log scale.

Cross-runtime comparison.

Table 3 and Figure 7 compare five runners where equivalent operations exist: Python NLTK 3.10.3, `bun_nltk` native, `bun_nltk` WASM, Node.js 24 with WASM, and npm `natural`. We use two warmups and the median of five wall-clock rounds on a MacBook Pro M5 Pro arm64. Datasets derive from the `movie_reviews` corpus (1MB prose, 100k word list, 1,000 labeled documents). N/A marks unavailable operations. Some populated cells still use reduced work, as the table notes specify, so we exclude those cells from like-for-like speedup claims.



Top 24 of 46 families shown; import-name coverage only; source: docs/PARITY_CHECKLIST.md (241 indexed names)

Fig. 6 Import-name coverage for the 24 largest of 46 indexed families. Equal bar lengths show that every documented name in each displayed family imports; they do not distinguish the 210 functional entries from the 31 import-only entries.

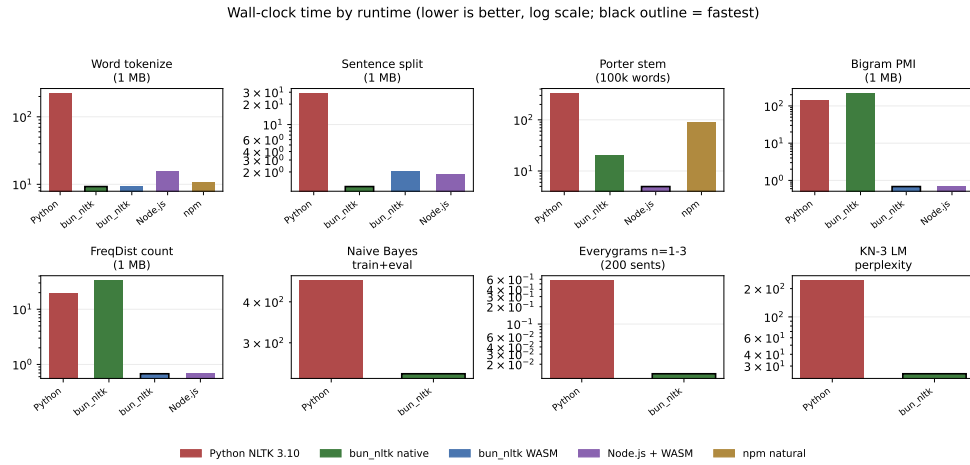


Fig. 7 Eight benchmark rows across five runtimes (log scale; black outline marks the smallest reported median). Missing cells denote unavailable operations. Reduced-workload Node and counting-only WASM cells are shown for engineering context but are excluded from like-for-like comparisons. Punkt is about 65% slower in WASM than native in this experiment.

Table 2 Speedups versus Python NLTK (Bun native). Median of 2 to 4 rounds on `gate_synthetic.txt`. See `artifacts/bench-dashboard.json`.

Task	Speedup	Notes
Token + n-gram counting	5.16×	1.15 M tokens
WASM n-gram (same task)	4.80×	WASM path, about 7% behind native
Collocations (PMI, top 30)	12.58×	Bigram PMI via Rust window
Porter stemmer (ASCII)	12.62×	Lowercasing + stemming
Sentence split (legacy)	4.49×	Naive splitter
Punkt sentence segmenter	19.75×	Trainable Punkt
Perceptron tagger	5.18×	about 1.15 M tagged tokens
Kneser-Ney LM (interpolated)	13.01×	Perplexity parity tolerant
Chunker	104.66×	Regex + NE chunk
WordNet lookup	101.25×	Dictionary lookup
Chart parser	37.26×	CFG / chart
Left-corner parser	34.69×	
Feature chart parser	147.08×	
Feature Earley	193.25×	
Naive Bayes	1.95×	Feature-dict wrapper
Linear classifier	253.75×	
Decision tree	10.14×	
Earley / PCFG	18.56/18.15×	
MaxEnt (GIS)	9.33×	
Cond. exponential / Pos. NB	19.52/1.97×	

Table 3 Wall-clock milliseconds by runtime (median of 5; M5 Pro, macOS arm64). Bold marks each row’s fastest runner. Generated by `paper/bench/run_bench.ts`; raw numbers in `paper/bench/results.json`.

Task	Python	Native	WASM	Node+WASM	natural
Word tokenize (1 MB)	223.7	9.3	9.4	15.6	10.6
Sentence split (1 MB)	29.0	1.2	2.0	1.8	N/A
Porter stem (100k)	331.7	20.2	N/A	5.0 ^a	92.2
Bigram PMI (1 MB)	145.2	219.8	0.7	0.7	N/A
FreqDist count (1 MB)	19.7	33.4	0.7	0.7	N/A
Naive Bayes train+eval	465.0	241.4	N/A	N/A	N/A
Everygrams $n=1..3$	0.57	0.01	N/A	N/A	N/A
KN-3 LM perplexity	244.6	24.8	N/A	N/A	N/A

^a Node stems 20k words through the WASM morphy path, while other populated cells stem 100k words with Porter. This cell is not comparable.

The WASM bigram and FreqDist cells expose counting only, while the Python and native bigram cells include PMI scoring and candidate materialization. They are throughput demonstrations, not like-for-like speedups.

The everygrams value rounds a sub-millisecond duration to two decimals; it is near the timer and display granularity and should not be read as 0.01 ms precision.

Native word tokenization, Punkt, and Porter take less time than their Python rows. WASM overhead varies: word tokenization is close to native, while Punkt rises from 1.2 to 2.0 ms. The 0.7 ms WASM counting cells do less work than the Python and native PMI path, so they cannot support a $212\times$ end-to-end claim. Native is slower than Python on collocation scoring and FreqDist because the wrapper materializes candidate pairs and JavaScript objects. These timings measure the implementations, not algorithmic equivalence.

Scaling with input size and real corpora.

The fixed-size comparison above could hide constant-overhead effects, so we re-ran three tasks (word tokenize, Punkt sentence split, bigram PMI) over the same prose grown from 10 KB to 10 MB, and separately over two classic corpora: the Brown corpus (1,440,413 tokens) and Gutenberg literary texts (Milton’s *Paradise Lost*, Austen’s *Emma*, Melville’s *Moby Dick*; 543,081 tokens). Figure 8 plots all three runners on log-log axes; Table 4 lists the corpus numbers.

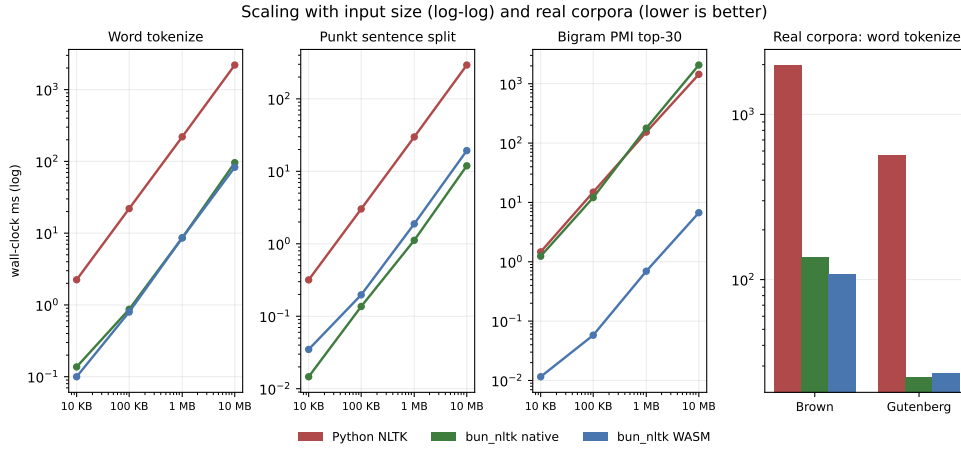


Fig. 8 Left to right: word tokenize, Punkt split, and bigram work as input grows 10 KB $\rightarrow$ 10 MB (log-log); the rightmost panel is word tokenization over Brown and Gutenberg. Similar slopes show that measured runtime differences persist across these input sizes. They do not prove algorithmic equivalence. The bigram panel also mixes counting-only WASM with full scoring paths, so those lines are not directly comparable.

The scaling study shows how runtime changes with input size; it does not establish output equivalence. Tokenize speedups remain between $16.4\times$ and $25.5\times$ over 10 KB to 10 MB, while Punkt remains between $22\times$ and $27\times$. On Brown, tokenization drops from 2.0s in Python to 137 ms native; on Gutenberg it is $21.9\times$ faster. The bigram rows require a different reading: the WASM export counts only, whereas Python and native run the full PMI top-30 pipeline. The real-corpus FreqDist rows also compare a native JavaScript-object wrapper with a counting-only WASM export. We report those timings as implementation measurements, not algorithmic equivalence.

Table 4 Real popular corpora, wall-clock ms (median of 5; same machine). Brown: 1.44M tokens; Gutenberg: 543k tokens. Reuters was not present in the local NLTK data directory and is reported as skipped rather than estimated. Raw numbers in `paper/bench/scaling_results.json`.

Corpus	Task	Python	Native	WASM
Brown (1.44M tok.)	Word tokenize	1988.5	136.6	108.5
Brown	Punkt split	237.8	16.8	43.6
Brown	FreqDist	254.2	381.9	86.1
Gutenberg (543k)	Word tokenize	565.5	25.8	27.0
Gutenberg	Punkt split	83.0	4.4	6.9
Gutenberg	FreqDist	71.5	105.1	22.9

Throughput and resources.

On the token path we see about 15.5 M tokens per second and 0.67 M sentences per second. The tagger does roughly 1.20 M tagged tokens per second. Peak RSS delta is about 908 MB (corpus loaded) with roughly 187 MB heap. That is the dataset and the dictionary talking, not the 91.5 KiB WASM payload.

Correctness checks and known gaps.

The dashboard records sampled comparison flags, but these are not proofs of full equivalence. Its Punkt and sentence-count checks are false because Python and TypeScript segment the synthetic text differently. We treat this as a fidelity gap, not measurement noise. The 432-test suite checks selected contracts and examples. The separate 37-group differential gate checks live NLTK outputs or versioned NLTK-derived fixtures, while `PARITY_CHECKLIST.md` checks import names only. None establishes complete behavioral equivalence across all inputs.

Worked examples (listings as figures).

Listings 3 and 4 are minimal runnable examples from `examples/`. Both run on Bun, Node, and browsers through WASM.

Listing 3 Example: CCG chart parse “I sleep” $\Rightarrow$ S (`examples/ccg_quickstart.ts`).

```
import { fromString } from "bun_nltk/src/ccg_lexicon.ts";
import { CCGChartParser, DefaultRuleSet } from "bun_nltk/src/ccg_chart.ts";

const lex = fromString(`
:- S, NP, N
NP :: NP
N  :: N
I   => NP
sleep => S\NP
`);
const parser = new CCGChartParser(lex, DefaultRuleSet);
const parses = parser.parse(["I", "sleep"]);
console.log(parses.length, parses[0].lhs().toString());
// => 1 S
```

Listing 4 FOL resolution example (`examples/inference_resolution.ts`).

```
import { LogicParser } from "bun_nltk/src/sem_logic.ts";
import { ResolutionProverCommand } from "bun_nltk/src/inference_resolution.ts";

const lp = new LogicParser();
const assms = [lp.parse("all x.(man(x) -> mortal(x))"), lp.parse("man(socrates)")];
const goal = lp.parse("mortal(socrates)");
const cmd = new ResolutionProverCommand(goal, assms);
console.log(cmd.prove()); // => true
console.log(cmd.proof());
```

There are more examples for the pipeline (`ccg_quickstart`), tableau, discourse, and CHRF/BLEU scoring. All of them run with `bun run examples/*.ts`.

6 Case studies: end-to-end pipelines

Benchmarks isolate kernels, while applications chain them. Four scripts under `paper/usecases/` exercise complete pipelines. They print the outputs reproduced below from deterministic runs of the release build.

6.1 Sentiment classification over movie reviews

The sentiment pipeline follows the classic NLTK tutorial: load 1,000 labeled movie reviews, extract the top 2,000 word-presence features, train a Naive Bayes classifier, and evaluate on 200 held-out documents. The corpus has 500 positive and 500 negative reviews. A Fisher-Yates shuffle uses a Mulberry32 generator with seed 1337 before the 80/20 split. The seeded `bun_nltk` run reaches 78.5 % holdout accuracy in about 240 ms. The corresponding Python NLTK baseline scores 72.1 % with its seeded random split and 80.5 % with a label-sorted split. The package result lies inside that split-sensitive range and does not establish an accuracy improvement.

Listing 5 Captured output of `paper/usecases/sentiment_pipeline.ts` (abridged).

```
== Sentiment pipeline: Naive Bayes over movie reviews ==
Corpus source : ~/nltk_data/corpora/movie_reviews
Documents      : 1000 (500 pos / 500 neg)
Vocabulary     : 2000 word features
Split seed     : 1337
Train/test     : 800/200 documents
Holdout accuracy: 78.50% (157/200)

Predictions for unseen reviews:
| # | review                               | prediction | p_pos | p_neg |
| 1 | an absolute delight ...              | pos        | 0.6102 | 0.3898 |
| 2 | two hours of my life ...             | neg        | 0.0034 | 0.9966 |
| 3 | gorgeous cinematography...          | neg        | 0.1147 | 0.8853 |

Timing:
feature extraction + training: 193.5 ms
evaluation (200 docs): 43.8 ms
total wall time: 240.0 ms
```


The classifier exposes posterior probabilities without calibration. Review 3 receives $p_{\text{neg}} = 0.89$ because negative features dominate its word-presence vector; that number should not be interpreted as calibrated confidence.

6.2 Information extraction

The information-extraction script applies perceptron POS tagging, regular-expression chunking, and named-entity resolution to a paragraph about a corporate meeting. Tagging 22 tokens takes 1.5ms and chunking adds 0.6ms. The rule-based named-entity approximation finds Tim Cook and Satya Nadella as PERSON, but assigns GPE to Microsoft, Apple, OpenAI, and Redmond:

Listing 6 Captured output of `paper/usecases/information_extraction.ts` (abridged).

```
Named entities found:
| # | type | text |
| 1 | PERSON | Tim Cook |
| 2 | GPE | Microsoft |
| 3 | GPE | Redmond |
| 4 | PERSON | Satya Nadella |
| 5 | GPE | Apple |
| 6 | GPE | OpenAI |

IOB tags:
Tim/NNP/B-PERSON Cook/NNP/I-PERSON visited/VBD/O
Microsoft/NNP/B-GPE ... Satya/NNP/B-PERSON Nadella/NNP/I-PERSON ...

Timing: perceptron POS tagging (22 tokens): 1.50 ms
        NE chunking: 0.56 ms    total wall time: 2.06 ms
```

This output preserves the tree and IOB data shape, not NLTK's labels. Python NLTK labels Microsoft and OpenAI as ORGANIZATION, Apple as PERSON, Tim Cook and Satya Nadella as PERSON, and Redmond as GPE on the same sentence. The default `bun_nltk` grammar folds unmatched singular proper nouns into GPE, a documented fidelity gap.

6.3 WordNet semantics

The WordNet script queries the noun *car*. It prints all five synsets with their lemmas and glosses, follows the motor-vehicle hypernym chain to *entity*, and computes path similarities for selected word pairs. The optional packed image contains 117,659 synsets and occupies 30,602,237 bytes, about 30 MB. We generate it from the official database and load it through `BUN_NLTK_WORDNET_PATH`. The npm tarball omits this image:

Listing 7 Captured output of `paper/usecases/wordnet_query.ts` (abridged).

```
Synsets of 'car': 5 found
Focus sense: auto, automobile, car, machine, motorcar (02958343.n)
Gloss: "a motor vehicle with four wheels; usually propelled by
       an internal combustion engine"

Hypernym chain: auto -> automotive_vehicle -> self-propelled_vehicle
                -> wheeled_vehicle -> container -> instrumentality
                -> artefact -> unit -> object

Semantic similarity (path similarity):
| pair          | path_similarity | lowest_common_hypernym |
| car ~ automobile | 1.0000         | auto                   |
| car ~ bicycle   | 0.3333         | wheeled_vehicle        |
| car ~ banana    | 0.0833         | unit                   |

Timing: all WordNet queries: 108.77 ms (load) + queries
```

6.4 Translation-metric agreement

The translation script scores NLTK doctest sentence pairs with BLEU, GLEU, METEOR, chrF, and RIBES. Each result agrees with its Python counterpart to the published precision reported in Section 5. The metrics do not always rank the pairs alike:

Listing 8 Captured output of `paper/usecases/translate_eval.ts` (abridged).

```
Scores (1.0 = perfect match):
| pair | BLEU | GLEU | METEOR | chrF | RIBES |
| BLEU | 0.5325 | 0.4394 | 0.6471 | 0.2780 | 0.3376 |
| METEOR | 0.4490 | 0.4394 | 0.6471 | 0.5452 | 0.3376 |
| RIBES | 0.7825 | 0.7826 | 0.9894 | 0.9514 | 0.9802 |

Timing: all metrics for 3 pairs: 2.70 ms
```

On the near-paraphrase pair, RIBES rewards the preserved word order (0.98) while chrF penalizes character-level divergence less than BLEU does; on the doctest pair, METEOR's synonym matching lifts it above BLEU. All five metrics complete in under 3 ms, which makes them practical inside a JavaScript test suite for continuous MT evaluation.

7 Discussion

Distribution through npm and WASM.

The npm registry contains about two million packages and is the default distribution channel for JavaScript. `npm install bun_nltk` installs the TypeScript sources, compact models, native prebuilts, and WASM without invoking a compiler or Python.

The explicit `WasmNltk` API supports hosts that forbid `dlopen` [15]. The committed browser tests pass in Chromium and Firefox. We have not recorded results for Safari, mobile browsers, or edge providers, so compatibility with those hosts remains unverified.

Known limits.

The inventory contains 31 import-only entries: five package barrels and 26 shim modules. The `app`, `draw`, and `twitter` groups consist entirely of shims. `chat` mixes working re-exports with unsupported operations, and `inference.prover9/mace` requires external binaries. Unsupported calls throw `Error("... requires ...; use JS alternative ...")` and name an alternative when one exists. Tests check that package-specific contract. Importability does not imply Python-equivalent behavior.

Rebuilding all four binaries on Apple Silicon requires Rust, the `wasm32-unknown-unknown` target, Zig, and `cargo-zigbuild` [14]. Package users receive prebuilds and do not compile Rust during installation.

The optional full WordNet dictionary is 30,602,237 bytes, about 30 MB, while the WASM binary is 93,712 bytes. The npm package ships mini and extended WordNet JSON files but excludes the full pack. The memory peak in Table 2 includes the loaded data as well as executable code.

When to use `bun_nltk`.

The package is suitable when its documented functional subset and measured fidelity fit the task. Code that depends on exact Punkt segmentation, NLTK’s statistical named-entity labels, Tk GUIs, Twitter access, or bundled `prover9/mace` behavior needs separate validation or another implementation.

8 Separate roles of WASM and npm

8.1 WASM as an execution target

WASM runs in browser tabs, WebWorkers, ServiceWorkers, and edge runtimes that forbid native code [12, 15]. The 91.5 KiB module supports client-side and offline tools without a Python process. Download and startup time depend on the network, host, cache, and surrounding JavaScript, and this study does not measure deployment latency. Chromium and Firefox pass the browser tests. Safari, mobile browsers, and edge providers remain untested.

8.2 npm as a distribution channel

The npm tarball contains ESM imports, focused subpath exports, selected models, and native/WASM artifacts under one version. `docs/API.md` documents the API. The release command runs 432 tests and the 37-group differential gate locally, without a hosted CI account.

WASM supplies the execution target; npm supplies the package. Neither removes the need to test each host.

9 Threats to validity

Construct validity.

The 241-entry denominator comes from module headings scraped from the NLTK API documentation on 2026-08-24. It measures import-name coverage, not every symbol or behavior, and the denominator may change with the documentation. We therefore report 210 functional entries separately from 31 import-only entries. Five of the latter are barrels and 26 are genuine shims. The author wrote both the implementation and the differential gate, which creates a circularity risk even though Python generates many expected outputs at run time. External test suites and downstream replications are still needed.

Internal validity.

Table 2 comes from a recorded single-core Linux x64 environment and reports medians of 2 to 4 rounds without variance or confidence intervals. Tables 3 and 4, plus Fig. 8, come from one MacBook Pro M5 Pro arm64 with five measured rounds and two warmups. Table 1 uses 15 rounds per implementation on the same Mac and reports deterministic 95% percentile-bootstrap intervals. Fifteen samples still give limited tail information, and sequential native/TypeScript runs may retain order effects. We keep the Linux and Mac environments separate. Some cross-runtime cells perform reduced work: Node uses morphy on 20k words rather than Porter on 100k, and two WASM cells count without completing the Python/native scoring pipeline. Timer granularity also limits the 0.01 ms everygrams row. The reported local versions are Bun 1.4.0, Node 24.19.0, CPython 3.12.13, and NLTK 3.10.3.

External validity.

The browser harness exercises Chromium and Firefox, but no browser result JSON is committed. Safari, mobile browsers, and edge providers remain untested. The npm release includes native binaries for macOS arm64, Linux x64, and Windows x64. The packaged macOS binary has run on the author’s machine. Clone-and-run validators are provided for native x64 Linux and Windows hosts, but their external reports remain outstanding at the time of writing. We do not substitute Wine, cross-compilation, or an emulated architecture for those results. The English synthetic data and selected corpora do not represent every language, domain, input encoding, or deployment host.

Conclusion validity.

Runtime speed and behavioral fidelity are separate claims. Similar scaling slopes do not establish algorithmic equivalence, and import coverage does not establish functional coverage. Punkt segmentation differs on the dashboard sample, and the rule-based named-entity example disagrees with Python NLTK on Microsoft, Apple, and OpenAI. The benchmark comparisons support only the workloads, implementations, and environments described here.

10 Conclusion

We built a TypeScript NLP toolkit with import coverage for all 241 names in a frozen NLTK documentation index. The inventory contains 210 functional entries, 26 genuine shims, and five import-only package barrels. A 432-test suite and a separate 37-group differential gate check selected contracts, while the paper records known Punkt and named-entity fidelity gaps. On the recorded Linux benchmark, selected native tasks range from $12\times$ to $253\times$ faster than Python. The newer 15-round M5 Pro comparison measures $1.80\times$ to $2.54\times$ median speedups for native text training, HMM decoding, K-means, and WordNet loading. Their 95% intervals remain above $1\times$. The WordNet hot-batch interval crosses $1\times$, so that result is inconclusive.

GUI, network, and external-binary operations remain shims. The optional full WordNet payload is about 30 MB, and rebuilding all targets needs Rust and Zig. Browser validation does not yet cover Safari, mobile, or edge hosts. Native Linux and Windows package reports still need to be collected from those operating systems.

The immediate release task is to collect and archive the native Linux and Windows reports. External fidelity tests, Safari and edge-host runs, a versioned model registry, and a public API-index diff remain open work.

Artifact availability.

Code: https://github.com/Seyamalam/bun_nltk (v0.16.0, Apache-2.0). Package: https://www.npmjs.com/package/bun_nltk. WASM: `native/bun_nltk.wasm` (93,712 bytes). Raw benchmark JSON and deterministic data-generation seeds are committed under `artifacts/`, `paper/bench/`, and `bench/generate_synthetic.py`. `paper/bench/native_migration_results.json` records the 15-round timings, raw samples, and bootstrap intervals. `artifacts/fidelity-report.json` records the 37 behavioral groups and versions. `bun run release:local` reproduces the author’s local package validation. Native x64 Linux and Windows testers run `scripts/verify-linux.sh 15` and `scripts/verify-windows.cmd -Rounds 15`; each command writes a JSON report for archival. We will archive the accepted artifact and add a Zenodo DOI when the paper is accepted.

References

- [1] Bird, S., Klein, E., Loper, E.: Natural Language Processing with Python. O’Reilly Media, Sebastopol, CA (2009). <https://www.nltk.org/book/>
- [2] Loper, E., Bird, S.: NLTK: The natural language toolkit. In: Proc. ACL-02 Workshop on Effective Tools and Methodologies for Teaching NLP and CL, Philadelphia, Pennsylvania, USA, pp. 63–70 (2002). Association for Computational Linguistics. <https://aclanthology.org/W02-0109/>
- [3] Natural Project: natural: General natural language facility for Node.js. <https://github.com/NaturalNode/natural>. npm `natural`, MIT (2024)

- [4] Wink JS: winkNLP: High-performance JavaScript NLP. <https://winkjs.org/wink-nlp/> (2024)
- [5] Kelly, S.: compromise: Natural language processing in JavaScript. <https://github.com/spencermountain/compromise> (2024)
- [6] AXA Group, Seijas, J.: nlp.js: An NLP library for building bots, with entity extraction, sentiment analysis and language detection. <https://github.com/axa-group/nlp.js>. npm `nlp.js`, Apache-2.0 (2024)
- [7] NLTK Project: NLTK 3.9 Documentation and API Index. <https://www.nltk.org/api/nltk.html>. Accessed 2026-08-24 (2024)
- [8] Honnibal, M., Montani, I.: spaCy: Industrial-strength Natural Language Processing. Explosion AI. <https://spacy.io> (2023)
- [9] Qi, P., Zhang, Y., Zhang, Y., Bolton, J., Manning, C.D.: Stanza: A python natural language processing toolkit for many human languages. In: Proc. ACL System Demonstrations, Online, pp. 101–108 (2020). Association for Computational Linguistics. <https://aclanthology.org/2020.acl-demos.14/>
- [10] Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., Drame, M., Lhoest, Q., Rush, A.M.: Transformers: State-of-the-art natural language processing. In: Proc. EMNLP System Demonstrations, Online, pp. 38–45 (2020). Association for Computational Linguistics. <https://aclanthology.org/2020.emnlp-demos.6/>
- [11] Oven Inc.: Bun: All-in-one JavaScript Runtime. <https://bun.sh>. `bun:ffi dlopen`, backed by JavaScriptCore (2024)
- [12] WebAssembly Working Group: WebAssembly Core Specification, Version 2.0. W3C Recommendation, June 2023. <https://www.w3.org/TR/wasm-core-2/> (2023)
- [13] Haas, A., Rossberg, A., Schuff, D.L., Titzer, B.L., Holman, M., Gohman, D., Wagner, L., Zakai, A., Bastien, J.F.: Bringing the web up to speed with webassembly. In: Proc. PLDI, pp. 185–200 (2017). ACM. <https://doi.org/10.1145/3062341.3062363>
- [14] Klabnik, S., Nichols, C.: The Rust Programming Language, 2nd edn. No Starch Press, San Francisco, CA (2023). <https://doc.rust-lang.org/book/> (online ed. continuously updated)
- [15] Cloudflare, Vercel: Cloudflare Workers and Vercel Edge Runtime: WASM-only code on the edge. <https://workers.cloudflare.com/> and <https://vercel.com/docs/functions/edge-functions> (2024)